A Deep Dive into Rainbow DQN

Chapter 1: Double Q-Learning (DDQN)

Harsh Verma

June 26, 2025

Abstract

Deep Q-Networks (DQN) revolutionized the field of reinforcement learning but suffer from a subtle yet significant flaw: a systematic overestimation of action-values. This phenomenon, known as maximization bias, can lead to suboptimal policies and unstable training. This chapter dissects the root cause of this bias, introducing the Double DQN (DDQN) algorithm as an elegant and effective solution. We will explore the intuition, mathematical theory, and practical implementation of DDQN, establishing it as a foundational component of the Rainbow agent.

1 Foundations: Value-Based Learning and its Pitfalls

1.1 The Bellman Optimality Equation

At the heart of Q-learning lies the Bellman equation. For an optimal policy, the action-value function $Q^*(s, a)$—which represents the maximum expected discounted future return achievable from state s by taking action a—must satisfy a specific consistency condition:

$$Q^*(s, a) = \mathbb{E}\left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a')\right]$$

This equation asserts that the value of the current state-action pair is the expected immediate reward (r_{t+1}) plus the discounted value of acting optimally from the next state (s_{t+1}).

1.2 The Genesis of Overestimation Bias

In DQN, we use a neural network $Q(s, a; \theta)$ as a function approximator for Q^*. The network is trained by minimizing the error against a target, $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$, derived from this equation. The problem originates from the max operator when applied to the network's value estimates, which are inherently noisy.

1.2.1 An Intuitive Analogy

Imagine you want to find the tallest person in a room, but you cannot measure them directly. Instead, for each person, you get five different noisy measurements of their height. To find the "tallest" person, you could take the highest measurement you got for each person and then pick the person with the highest of these maximums. This strategy will almost always lead you to believe the tallest person is taller than they actually are, because you are systematically selecting for positive measurement errors. The DQN target calculation does exactly this with Q-values.

1.2.2 The Mathematical Root: Jensen's Inequality

This observed bias is a direct consequence of Jensen's Inequality. For a convex function f (like max) and a random variable X, the inequality states that $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$. In our context:

$$\mathbb{E}\left[\max_{a'} Q(s', a'; \theta^-)\right] \geq \max_{a'} \mathbb{E}\left[Q(s', a'; \theta^-)\right]$$

The term on the left is what DQN computes for its target—the maximum of noisy estimates. The term on the right is the true value of the optimal action—the maximum of the (unknown) true expected values. This inequality guarantees that the DQN target is, on average, an overestimation of the true value. This can cause the agent to converge to poor policies that it was "tricked" into believing were optimal.

2 The Double DQN Algorithm: Decoupling for Accuracy

2.1 The Core Idea: Breaking the Maximization Loop

The solution proposed by Hasselt et al. (2016) is to break the flawed process where the same network values are used to both **select** an action and **evaluate** that action. Double DQN decouples these two steps by assigning them to two different networks.

2.2 The Mechanism: A Tale of Two Networks

- **Role of the Online Network (θ):** This is the primary, most up-to-date network. Its job is to act as the *policy driver*. It is used exclusively to select which action it currently believes is the best for the next state.
- **Role of the Target Network (θ^-):** This is the older, more stable network. Its job is to act as the *value estimator*. It provides an evaluation for the action that was proposed by the online network.

Why this works: Because the online and target networks have different weights, their estimation errors are decorrelated. It is statistically unlikely that both networks will have a large positive error for the same suboptimal action. The online network might select an action due to an optimistic error, but the target network's evaluation of that same action is likely to be more sober and accurate, thus correcting the bias.

3 Formalism and Implementation

3.1 Mathematical Definition

The subtle yet profound change is best seen by comparing the target equations directly.

Standard DQN Target:

$$y_t^{\text{DQN}} = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) \tag{1}$$

Double DQN Target:

$$y_t^{\text{DDQN}} = r_t + \gamma \underbrace{Q\left(s_{t+1}, \overbrace{\arg\max_{a'} Q(s_{t+1}, a'; \theta)}^{\text{1. Select action with online network}}, \theta^-\right)}_{\text{2. Evaluate that action with target network}} \tag{2}$$

In Equation 2, the online network θ first finds the best action a^*, and then the target network θ^- provides the value estimate for that specific a^*.

3.2 Pseudocode Algorithm

The change only affects the target calculation within the agent's learning step.

```
# Function learn(replay_buffer):
#   1. Sample a minibatch of transitions (s, a, r, s') from replay_buffer
#   2. Compute the next actions a* using the online network:
#      a_star = argmax(online_net.predict(s'))
#
#   3. Get the value of these actions using the target network:
#      q_next = target_net.predict(s')[a_star]
#
#   4. Compute the DDQN target:
#      target = r + gamma * q_next
#
#   5. Compute loss between online_net.predict(s)[a] and target
#   6. Perform gradient descent step on online_net
```

Listing 1: Pseudocode for a single learning step with DDQN.

Figure 1: A simplified algorithm for a DDQN learning update.

3.3 PyTorch Implementation Analysis

The following snippet demonstrates the practical implementation of the target calculation logic.

```
# online_net and target_net are the two neural networks.
# next_states is a tensor of next states from the replay buffer.

with torch.no_grad(): # We don't need gradients for the target calculation.

    # Step 1: Select best action with the online network.
    # Its output is the INDEX of the best action for each state in the batch.
    best_next_actions = online_net(next_states).argmax(dim=1, keepdim=True)

    # Step 2: Get the Q-values for all next actions from the target network.
    target_next_q_values = target_net(next_states)

    # Step 3: Evaluate the chosen actions using the target network's values.
    # The .gather() method efficiently selects the Q-value from each row
    # of target_next_q_values corresponding to the action index in
    best_next_actions.
    q_value_of_best_action = target_next_q_values.gather(1, best_next_actions)

    # Step 4: Compute the final DDQN target value.
    y_target = rewards + (1 - dones) * gamma * q_value_of_best_action
```

Listing 2: PyTorch implementation of the DDQN target calculation.

4 Empirical Evidence and Discussion

4.1 Visualizing the Impact

The original DDQN paper provides powerful visualizations of its effect:

- **Q-Value Plots:** When plotting the average predicted Q-value during training, the value for standard DQN often rises to unrealistic, pathologically optimistic levels. In contrast,

the values for DDQN remain much more subdued and closer to the actual range of achievable scores in the game, providing clear evidence of bias reduction.

- **Performance Plots:** Across the suite of Atari games, DDQN consistently achieves higher and more stable scores than its predecessor, demonstrating that learning a more accurate value function leads to a better final policy.

4.2 Synergy within Rainbow

DDQN's role extends beyond its standalone performance improvement. It is a crucial **stabilizing foundation** for other, more complex components within Rainbow.

- **Prioritized Experience Replay (PER):** PER relies on the TD-error to determine an experience's importance. By providing a more accurate target value, DDQN produces a more reliable TD-error. This allows PER to focus on genuinely surprising events, rather than transitions with high error due to simple value overestimation.
- **Distributional DQN:** Similarly, the distributional component learns a full distribution of returns. DDQN helps to correctly anchor the mean of this learned distribution, leading to a more stable and accurate representation of value uncertainty.